

Microsoft Copilot Studio

Day 2 — Conversational Design

Participant Handbook

Handbook detail	Value
Course	Microsoft Copilot Studio — Instructor-Led Training (ILT)
Day / Duration	Day 2 of the series — 1 full day (approx. 7 hours + breaks)
Delivery mode	Instructor-led, hands-on (classroom or virtual)
Audience	Makers who completed Day 1, business analysts, functional consultants, IT pros
Modules covered	Module 1 Topics, Module 2 Conversation Design, Module 3 Advanced Conversations + Labs 7 to 13
Prerequisite	Day 1 — Foundations and Getting Started, and a working agent from Labs 1 to 6
Product version	Microsoft Copilot Studio — classic experience (topic authoring) and the new agent experience (production-ready preview)
Document version	v1.0 — July 2026
Prepared by	Learning Team

© . For classroom use by registered participants only. Microsoft, Copilot Studio, Power Platform, Dataverse, Power Fx, Microsoft 365 and Azure are trademarks of the Microsoft group of companies. Product screens change frequently; always confirm against Microsoft Learn.

Before You Begin

Day 2 assumes you finished Day 1. You should already have an agent in your development environment, one knowledge source attached, and a published version you can reach in Teams or Microsoft 365 Copilot. You will extend that same agent today.

What Day 1 covered — and what Day 2 adds

Day 1 gave you	Day 2 gives you
Product, licensing, architecture and lifecycle	The mechanics of conversation itself
Environments, security roles, Dataverse	Topics, trigger phrases and intent recognition
Creating an agent, instructions, knowledge, publishing	Variables, entities, memory, conditions, loops and branching
Single-turn testing in the test canvas	Multi-turn design, adaptive responses, escalation and error handling
One simple agent	Two complete agents: an FAQ assistant and an employee leave assistant

NOTE

Day 2 works mainly in the classic authoring experience, because that is where the topic canvas, node types, variables and entities live. Where the new agent experience handles the same idea differently — for example Memory instead of global variables — the handbook says so explicitly and links to the right documentation.

Icons used in this handbook

Box	What it means
NOTE	Background information you should read.
TIP	A shortcut or field trick that saves time.
WATCH OUT	A common mistake. Read before you click.
BEST PRACTICE	How experienced makers do it in real projects.
[SCREENSHOT x.x]	Where the matching product screen goes.
[DIAGRAM x.x]	Where a concept diagram goes. Some diagrams are already drawn inline.

Contents

Before You Begin	2
What Day 1 covered — and what Day 2 adds	2
Icons used in this handbook	2
Contents	3
Day 2 at a Glance	6
Agenda	6
Checklist before the first lab.....	6
Module 1 — Topics	7
Learning Objectives	7
1.1 What a Topic Is.....	7
The node types you will use today	7
1.2 System Topics.....	8
1.3 Trigger Phrases	8
How to add them.....	8
What makes a good trigger phrase.....	8
1.4 Generative Topics	9
Writing a description the planner can use	9
1.5 Intent Recognition	10
Step-by-step Activity — Read your own agent (10 minutes).....	10
Best Practices	11
Common Mistakes	11
Knowledge Check.....	11
Key Takeaways	12
Learn More — Microsoft Learn and Official Documentation.....	12
Module 2 — Conversation Design.....	13
Learning Objectives	13
2.1 Variables	13
Passing values between topics	13
Power Fx in one page.....	13
2.2 Entities	14
Adding a question with an entity	14
2.3 Memory	15
2.4 Conditions.....	15
2.5 Loops	16
2.6 Switch Cases	16
Step-by-step Activity — Trace a value (10 minutes)	17
Best Practices	17
Common Mistakes	17
Knowledge Check.....	17

Key Takeaways	18
Learn More — Microsoft Learn and Official Documentation	18
Module 3 — Advanced Conversations	20
Learning Objectives	20
3.1 Multi-turn Conversations	20
3.2 Adaptive Responses	21
3.3 Escalation	21
3.4 Error Handling	22
Invalid answers — your three options	22
Runtime errors	22
3.5 Conversation Best Practices	23
Step-by-step Activity — Break your agent on purpose (10 minutes)	23
Best Practices	23
Common Mistakes	23
Knowledge Check	24
Key Takeaways	24
Learn More — Microsoft Learn and Official Documentation	25
Hands-on Labs — Day 2	26
Lab 7 — Create Topics	27
Scenario	27
Part 1 — A simple answer topic	27
Part 2 — A topic with a question	27
If something goes wrong	27
Lab 8 — Configure Trigger Phrases	28
Part 1 — Write good phrases	28
Part 2 — Break it on purpose	28
Part 3 — Write a description	28
Lab 9 — Build Multi-turn Conversations	29
Steps	29
Part 2 — Try an interruption	29
If something goes wrong	29
Lab 10 — Use Variables and Entities	30
Part 1 — A custom closed-list entity	30
Part 2 — A regular-expression entity	30
Part 3 — Pass a value between topics	30
If something goes wrong	30
Lab 11 — Implement Conditional Logic	31
Part 1 — The switch	31
Part 2 — A rule with Power Fx	31
Part 3 — A loop	31
Reflection	31

Lab 12 — Design FAQ Assistant..... 32

 Scenario 32

 Steps..... 32

 Your test results 32

Lab 13 — Build Employee Leave Assistant 33

 Scenario 33

 Steps..... 33

 Acceptance tests 33

 Reflection 34

Appendix A — Day 2 Vocabulary 35

Appendix B — Day 2 Quick Reference 36

 Node cheat sheet 36

 Power Fx snippets you will reuse 36

 Trigger phrase rules 36

 The five beats of a transaction 36

 Never dead-end checklist 36

Appendix C — Knowledge Check Answer Key..... 37

 Module 1 — Topics 37

 Module 2 — Conversation Design 37

 Module 3 — Advanced Conversations 37

Appendix D — Notes and Next Steps 38

 One conversation I will redesign at work 38

 Coming up on Day 3..... 38

NOTE
Page numbers update automatically. In Microsoft Word, press Ctrl+A then F9 and choose 'Update entire table'.

Day 2 at a Glance

Today is about control. Day 1 made an agent that answers; Day 2 makes an agent that holds a conversation — asks the right questions, remembers the answers, branches sensibly, recovers from mistakes and knows when to hand over to a human.

Agenda

Time	Session	Format
09:00 – 09:15	Recap of Day 1, questions, environment check	Discussion
09:15 – 10:30	Module 1 — Topics	Concept
10:30 – 10:45	Break	—
10:45 – 11:45	Lab 7 — Create Topics · Lab 8 — Configure Trigger Phrases	Hands-on
11:45 – 13:00	Module 2 — Conversation Design	Concept
13:00 – 14:00	Lunch	—
14:00 – 15:15	Lab 9 — Multi-turn · Lab 10 — Variables and Entities · Lab 11 — Conditional Logic	Hands-on
15:15 – 16:00	Module 3 — Advanced Conversations	Concept
16:00 – 16:15	Break	—
16:15 – 17:15	Lab 12 — FAQ Assistant · Lab 13 — Employee Leave Assistant	Hands-on
17:15 – 17:30	Recap, knowledge check review, Day 3 preview	Discussion

Checklist before the first lab

- You can sign in to Copilot Studio and see your development environment.
- Your Day 1 agent opens and its knowledge source still works.
- You know whether your agent uses generative orchestration or classic orchestration — Module 1 shows you where to check.
- You have a notepad or text file open to keep copies of trigger phrases and Power Fx expressions.

TIP

Keep your Day 1 handbook beside you. The interface map you completed in Lab 2 will save you time all day.

Module 1 — Topics

Duration: approximately 75 minutes. Related labs: Lab 7, Lab 8.

Learning Objectives

By the end of this module you will be able to:

- Explain what a topic is and describe the node types that make up a topic.
- Identify the system topics every agent has, and say what each one is for.
- Write trigger phrases that work, and explain why five good phrases beat fifty weak ones.
- Explain how generative topics differ from trigger-phrase topics, and write a topic description the orchestrator can use.
- Describe how intent recognition works, and what happens when the agent matches nothing or matches several topics.

1.1 What a Topic Is

A topic is a self-contained piece of conversation logic. Think of it as one job the agent knows how to do: check an order, book a room, explain the leave policy. Inside a topic you place nodes on a canvas, and the agent walks through those nodes when the topic is triggered.



Figure 1.1 — the shape of almost every topic: something triggers it, it gathers what it needs, it branches, it responds, it ends.

The node types you will use today

Node	What it does	Day 2 module
Trigger	Defines when the topic starts — a phrase, an agent decision, an event, or a redirect from another topic.	1.3, 1.4
Send a message	Says something to the user. Supports formatting, images, quick replies and Adaptive Cards.	3.2
Ask a question	Asks the user for something and stores the answer in a variable. Uses an entity to understand the reply.	2.1, 2.2
Ask with Adaptive Card	Collects several values at once through a form-style card.	3.2
Add a condition	Branches the conversation based on a variable or a Power Fx expression.	2.4, 2.6
Loop through items	Repeats the same steps once for every item in a list or table.	2.5
Set a variable value	Stores or calculates a value, usually with Power Fx.	2.1
Add a tool / call an action	Runs a flow, a connector or a prompt.	Day 3
Redirect to another topic	Hands the conversation to another topic and, by default, comes back afterwards.	3.1
End current topic / End all topics	Finishes this topic, or finishes everything and returns to the top.	3.1

[SCREENSHOT 1.1]

The topic authoring canvas with a trigger node, a question node and a condition node connected.

TIP

Every topic also has a code editor. Open More, then Open code editor, to see the topic as YAML. It is the fastest way to copy a topic between agents, and the fastest way to review a change with a colleague.

1.2 System Topics

Every agent is created with a set of system topics already in place. You do not have to build them, but you should read them, because they decide what your users experience when things go off the happy path.

System topic	What it does	When it runs
Conversation Start	Greets the user and introduces what the agent can do.	When a conversation begins.
Conversational boosting	Produces generative answers from your knowledge sources. Created automatically when the agent is created.	When no other topic matches the question.
Fallback	Tells the user their question could not be matched and asks them to try again.	When the agent cannot match the message to any topic.
Multiple Topics Matched	Asks the user which of several topics they meant. Often called 'did you mean'.	When a message closely matches more than one topic.
Escalate	Tells the user they need a human, and hands off if a handoff is configured.	When 'talk to agent' matches, or when the Escalate event is called.
End of Conversation	Confirms the query was answered and closes politely.	When another topic redirects to it.
On Error	Tells the user an error occurred and shows an error code, conversation ID and timestamp.	When a user-facing error happens.
Reset Conversation	Clears variables and starts again. A good worked example of Clear all variables.	When the user or a topic asks to reset.
Sign in	Handles authentication when the agent needs the user to sign in.	When a protected step is reached.
Start Over	Restarts the conversation from the beginning.	On user request.

BEST PRACTICE

Rewrite Conversation Start, Fallback and Escalate in your own organisation's voice before you go live. The defaults are generic and they are the first and last thing users see.

Open Reset Conversation and read it. It is a free worked example of clearing global variables correctly.

1.3 Trigger Phrases

In an agent that uses **classic orchestration**, the default topic trigger is **User says a phrase**. You supply a set of example phrases — words, keywords and questions a user is likely to type — and the agent uses natural language understanding to match incoming messages against them.

How to add them

1. Open the topic and select the Trigger node.
2. Type a phrase and press Shift+Enter to put the next one on its own line.
3. Press Enter to add them all. You can also upload a file containing a list of trigger phrases.
4. Save the topic and test it in the test canvas using words you did not type in.

[SCREENSHOT 1.2]

The Trigger node with several trigger phrases listed, and the option to upload a phrase list.

What makes a good trigger phrase

Do	Don't	Why
Write 5 to 10 realistic phrases	Write 60 near-identical variations	The model generalises. Volume of similar phrases adds noise, not accuracy.

Do	Don't	Why
Use whole sentences the way users type them	Use single keywords like 'leave'	Single keywords collide with other topics and weaken matching.
Vary length, wording and formality	Repeat the same structure with one word changed	Variety is what teaches the model the shape of the intent.
Include real misspellings and shorthand you see in support tickets	Invent phrases nobody says	Trigger phrases should mirror your actual users.
Keep each topic's phrases clearly distinct	Let two topics share overlapping phrases	Overlap causes the Multiple Topics Matched topic to fire, or the wrong topic to win.

Good trigger phrases for a 'Check leave balance' topic:

how many leave days do I have left
 what's my leave balance
 remaining casual leave
 do I still have annual leave this year
 leave balance check

Weak set (do not copy this):

leave
 leave balance
 balance leave
 my leave balance
 leave my balance

WATCH OUT

Trigger matching keeps running while a user is inside a topic. If a message the user types mid-conversation matches another topic's trigger phrases, the agent can jump. Keep phrases distinct across topics, and see section 3.1 for how to handle interruptions deliberately.

1.4 Generative Topics

Newly created agents use **generative orchestration** by default. In this mode the agent does not match trigger phrases. Instead it reads the **name and description** of every topic, tool, knowledge source and connected agent, and a planning layer chooses the best combination to satisfy the request. The default trigger for a topic becomes **The agent chooses**.

	Classic orchestration	Generative orchestration
Default trigger	User says a phrase	The agent chooses
What drives selection	Trigger phrases and natural language understanding	Topic name plus the description on the Trigger node
Multi-step requests	One topic at a time; you script the chaining	The planner can combine topics, tools and knowledge in one plan
Filling topic inputs	You ask for each value with Question nodes	Inputs can be filled from conversation context, or the agent generates the question
Best for	Regulated, auditable, exactly-repeatable flows	Flexible assistants where users phrase things unpredictably

NOTE

If you switch an existing classic-orchestration agent to generative orchestration, Copilot Studio generates a default description for each topic from its trigger phrases. Those generated descriptions usually work, but they are a starting point — rewrite them.

Writing a description the planner can use

A description is not marketing copy. It tells the planner three things: what the topic does, when to use it, and what it needs.

Weak: "Leave topic"

Better: "Use this topic when an employee wants to check how many leave days they have remaining. Requires the employee's leave type (casual, sick or annual). Do not use this topic for applying for leave or cancelling a request."

BEST PRACTICE

Say explicitly what the topic is NOT for. Negative scope is the fastest way to stop the planner picking the wrong topic. Give every topic input a clear name and description, so the agent can fill it from context or ask a sensible question. Curate the toolkit. A small set of high-quality, non-overlapping topics beats an exhaustive set with fuzzy boundaries. Remove or disable topics you know are irrelevant — they only confuse the planner.

[SCREENSHOT 1.3]

A topic Trigger node in generative orchestration, showing 'The agent chooses' and the description field.

1.5 Intent Recognition

Intent recognition is the step between the user pressing Enter and your topic starting. Understanding it turns 'the bot is being weird' into a diagnosable problem.



Figure 1.2 — every message goes through the orchestrator before anything runs.

What happened	What the agent does	What you should change
One topic clearly matches	That topic runs.	Nothing.
Several topics match closely	The Multiple Topics Matched system topic asks the user which they meant, and sets a system variable identifying the choice.	Sharpen the trigger phrases or descriptions so the topics stop overlapping.
Nothing matches	The Fallback system topic runs. In most agents this hands over to Conversational boosting, which answers from your knowledge sources.	Add trigger phrases, or accept that knowledge should answer it.
Nothing matches and there is no knowledge	The user gets a generic 'I did not understand' message.	Add knowledge, or add the missing topic.

[DIAGRAM 1.1]

Intent recognition decision tree: message → single match / multiple match / no match → topic, 'did you mean' prompt, or fallback and generative answers.

TIP

The fastest diagnosis tool is the test canvas trace. Turn on the option to track between topics, then send the failing message and watch which topic actually opened.

Step-by-step Activity — Read your own agent (10 minutes)

1. Open your Day 1 agent and go to the Topics page.
2. Note whether topics show trigger phrases or 'The agent chooses'. Write which orchestration mode you are in: _____
3. Open the Conversation Start system topic and read the greeting. Would a real user of your agent understand it?
4. Open the Fallback system topic and note what it currently says.
5. Open Conversational boosting and find the generative answers node inside it.

6. In the test canvas, type something deliberately unrelated to your agent and watch which system topic answers.

Best Practices

- One topic, one job. If a topic needs the word 'and' to describe it, split it.
- Name topics as verbs from the user's point of view: 'Check leave balance', not 'LeaveBalanceV2'.
- Write the description even in classic orchestration — it costs nothing and future-proofs the topic if you switch modes.
- Rewrite the system topics before go-live. They carry your organisation's tone.
- Keep a text file of every topic's trigger phrases so you can spot overlaps across topics at a glance.

Common Mistakes

Mistake	Why it hurts	Do this instead
Using single keywords as trigger phrases	They collide across topics and match messages you never intended.	Write realistic whole sentences.
Adding 50 trigger phrases per topic	Adds noise and makes overlap harder to see, with no accuracy gain.	Five to ten varied, realistic phrases.
Leaving generated topic descriptions untouched	The planner picks the wrong topic and you cannot tell why.	Rewrite descriptions, including what the topic is not for.
Building a topic for something knowledge already answers	Duplicate answers that drift apart over time.	Let Conversational boosting handle general Q&A; use topics for processes.
Ignoring Multiple Topics Matched	Users see a confusing 'did you mean' prompt on ordinary questions.	Treat every appearance of it as a signal that two topics overlap.

Knowledge Check

Answer these on your own first, then compare with the answer key in Appendix C.

Q1. In an agent that uses generative orchestration, what is the default topic trigger?

- A. User says a phrase
- B. The agent chooses
- C. Activity received
- D. Redirect only

Q2. Which system topic produces generative answers from your knowledge sources?

- A. Fallback
- B. Conversation Start
- C. Conversational boosting
- D. On Error

Q3. A user asks a question and the agent replies by asking which of two things they meant. Which system topic ran?

- A. Fallback
- B. Multiple Topics Matched
- C. Escalate
- D. Start Over

Q4. Which set of trigger phrases is better for a 'Reset password' topic?

- A. password, reset, reset password, password reset
- B. I forgot my password, how do I reset my password, I'm locked out of my account, need a new password please
- C. help, issue, problem, support
- D. RESET_PASSWORD, PWD_RESET

Q5. You switch a classic-orchestration agent to generative orchestration. What happens to your existing topics?

- A. They stop working and must be rebuilt.
- B. Copilot Studio generates a default description for each one, based on its trigger phrases.

- C. Their trigger phrases are deleted.
- D. They are converted into knowledge sources.

Key Takeaways

- A topic is one job, built from nodes on a canvas, with a trigger at the top.
- System topics already handle greeting, fallback, ambiguity, escalation and errors — read them and rewrite them in your voice.
- Classic orchestration matches trigger phrases; five to ten varied, realistic sentences beat long lists.
- Generative orchestration matches topic names and descriptions; the description, including what the topic is not for, is what drives selection.
- When behaviour surprises you, look at intent recognition first: single match, multiple match, or no match.

Learn More — Microsoft Learn and Official Documentation

Microsoft Learn training

- **Create topics in Microsoft Copilot Studio (module)** — <https://learn.microsoft.com/en-us/training/modules/power-virtual-agents-topics/>
- **Create agents in Microsoft Copilot Studio (learning path)** — <https://learn.microsoft.com/en-us/training/paths/create-extend-custom-copilots-microsoft-copilot-studio/>
- **Create agents with Microsoft Copilot Studio — Online Workshop** — <https://learn.microsoft.com/en-us/training/paths/power-virtual-agents-workshop/>

Official documentation

- **Create and edit topics** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/authoring-create-edit-topics>
- **Set topic triggers** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/authoring-triggers>
- **Manage topics** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/authoring-topic-management>
- **Use system topics** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/authoring-system-topics>
- **Orchestrate agent behavior with generative AI** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/advanced-generative-actions>
- **Apply generative orchestration capabilities (guidance)** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/guidance/generative-orchestration>
- **Knowledge sources summary** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/knowledge-copilot-studio>
- **Implement your Copilot Studio projects (guidance overview)** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/guidance/implement-overview>

Module 2 — Conversation Design

Duration: approximately 75 minutes. Related labs: Lab 9, Lab 10, Lab 11.

Learning Objectives

By the end of this module you will be able to:

- Create and name variables, and explain the difference between topic, global, system and environment scope.
- Choose the right entity for a question, and explain what slot filling means.
- Describe what Memory does in the new agent experience and how it differs from a variable.
- Build a Condition node that branches on a variable or a Power Fx expression.
- Use a loop to repeat steps once for each item in a list.
- Build a switch-style branch, and know when to use one instead of a chain of conditions.

2.1 Variables

A variable is a labelled box that holds a value during a conversation. When a Question node collects an answer, Copilot Studio creates a variable to store it. You can also set variables yourself with a Set a variable value node, usually using a Power Fx formula.

Scope	Prefix you will see	Lives for	Use it for
Topic	Topic.	The current topic only.	Anything you collect and use in one place — the answer to a single question.
Global	Global.	The whole conversation, across topics.	Values many topics need: employee ID, preferred language, selected department.
System	System.	Provided by the platform.	Read-only context such as conversation ID, user language, activity details.
Environment	Env.	Set per environment.	Configuration that changes between development, test and production — a support email, an API base URL.

[SCREENSHOT 2.1]

The Variable properties panel showing variable name, type, scope and the option to receive values from other topics.

Passing values between topics

A topic that collects one piece of information can be reused by many other topics. To make that work, you open the variable's properties and set the scope so it can receive values from, or return values to, other topics.

1. Open the topic that collects the value and select the Question node's variable.
2. In the Variable properties panel, under Topic (limited scope), select Receive values from other topics.
3. Save the topic.
4. In the calling topic, add a Redirect to another topic node and choose the topic you just edited.
5. Select Add input for destination topic, choose the receiving variable, and pass in the value from the current topic.

WATCH OUT

Variables based on Date and time, Duration, Multiple choice options, or a custom entity cannot be passed directly between topics. Convert the value to text first, pass the text, and convert back if you need to.

Power Fx in one page

Power Fx is the formula language shared across Power Platform. In Copilot Studio you will meet it in condition expressions and in Set a variable value nodes.

```

Text(Topic.LeaveType)           convert a choice to text
Concatenate("Hi ", Global.EmployeeName)  join strings
IsBlank(Topic.EmployeeId)       true when nothing was captured
Lower(Trim(Topic.UserInput))    tidy up user input before comparing
DateDiff(Topic.StartDate, Topic.EndDate)  number of days between two dates
CountRows(Topic.LeaveRequests)  how many items in a table
If(Topic.Days > 5, "Manager approval", "Auto")  inline branching

```

TIP

A choice variable stored as an object shows up as [object Object] in a condition. Wrap it in Text() first — Text(Topic.YourChoice) = "Technical" — or set a new text variable before the condition.

BEST PRACTICE

Rename every auto-created variable straight away. 'Var1' is unreadable a week later; 'Topic.LeaveType' is obvious forever. Keep global variables to a short, documented list. Every global is shared state, and shared state is where bugs live. Put anything that differs between environments into an environment variable, never hard-coded in a message.

2.2 Entities

An entity tells the agent what kind of information to look for in the user's reply. When the agent finds it and stores it in a variable, that is called slot filling.



Figure 2.1 — slot filling: extraction plus storage in one step.

Entity kind	What it is	Example
Prebuilt	Ready-made entities Microsoft supplies — date and time, number, email, phone number, person name, city, money, duration, and many more.	"next Monday" recognised as a date.
Multiple choice options	The default for a Question node. You list the options and the agent shows them as buttons.	Casual / Sick / Annual.
Closed list (custom)	Your own fixed list of values, each with synonyms.	Department: Finance (accounts, fin), HR (people team).
Regular expression (custom)	A pattern for structured identifiers.	Employee ID in the form EMP-00000.
Options from a list variable	A dynamic entity: the options come from a table variable at runtime rather than being fixed.	Leave types returned from a system, shown as choices.

Adding a question with an entity

1. In your topic, add an Ask a question node.
2. Type the question the user will see.
3. Under Identify, choose what the agent should look for. Question nodes default to multiple-choice options; select a prebuilt or custom entity instead if you need one.
4. Rename the variable that stores the answer to something meaningful.
5. Open the node's properties and review what happens when the user gives an invalid answer — see section 3.4.
6. Save and test with a sentence rather than a single word, to confirm extraction works.

[SCREENSHOT 2.2]

An Ask a question node with the Identify list open, showing prebuilt entities and custom entities.

NOTE

Good extraction is why entities matter. If a user types 'I want to take three days of casual leave starting next Monday', well-chosen entities can fill the leave type, the number of days and the start date from that one sentence — no further questions needed.

BEST PRACTICE

Prefer a prebuilt entity over a custom one. Prebuilt entities already handle variations, formats and languages you have not thought of.

Give closed-list entities generous synonyms — write down how five different colleagues would say the same thing.

Use a regular expression entity for identifiers with a strict format, and give the user an example in the question text.

2.3 Memory

Variables live for one conversation. **Memory**, available in the new agent experience, lets an agent remember details across conversations. An agent with Memory turned on keeps a separate memory store for each user, so it can apply what it learned last week to today's request.

	Variables	Memory
Where you configure it	Inside a topic, on the authoring canvas.	The components panel in the new agent experience.
Lifetime	One session. Global variables last the conversation; everything resets afterwards.	Persists across conversations.
Scoped to	The conversation.	Each individual user, in a separate store.
Typical use	Values you collect and use in this exchange.	Preferences and context worth carrying forward — a usual office location, a preferred format.
How users control it	Not visible to them.	Users can ask the agent in chat what it remembers, update a memory, or ask it to forget something.
Status	Generally available.	Preview, in the new agent experience.

NOTE

Makers can delete all memory entries for an agent from the memory management portal — select the three dots, then Delete all memories. The first response from an agent with Memory turned on includes a link to that portal from the Preview tab.

WATCH OUT

Memory stores personal context by design. Confirm your organisation's data and privacy position before turning it on, and tell your users the agent remembers them.

[SCREENSHOT 2.3]

The Memory toggle in the components panel of the new agent experience, with the memory management portal alongside.

2.4 Conditions

A Condition node splits the conversation into branches. Each branch has a test; the first branch whose test is true is the one that runs. Everything that does not match falls into the 'All other conditions' branch.

[DIAGRAM 2.1]

Condition node with three branches: Casual, Sick and All other conditions, each leading to a different message.

1. In your topic, add a node and choose Add a condition.
2. Select the variable to test, choose the operator, and enter the value to compare against.

3. Select Add branch to add more tests. Order matters — the first true branch wins.
4. Use the 'All other conditions' branch for everything else. Never leave it empty.
5. For anything more complex than a single comparison, switch the branch to a formula and write Power Fx.

Simple: `Topic.LeaveType is equal to "Casual"`

Formula: `And(Topic.Days > 5, IsBlank(Topic.ManagerEmail))`

Formula: `Or(Text(Topic.Dept) = "Finance", Text(Topic.Dept) = "Legal")`

Formula: `DateDiff(Now(), Topic.StartDate, Days) < 3`

BEST PRACTICE

Always fill the 'All other conditions' branch with something useful — a clarifying question or a graceful message. An empty branch is a dead end for the user.

Test the boundary values, not just the middle. If the rule is 'more than five days', test four, five and six.

If a condition needs more than two clauses of Power Fx, set a well-named variable first and branch on that instead.

2.5 Loops

A loop repeats the same steps once for each item in a list or table. Use it when you have a collection — search results, leave requests, order lines — and you want to do the same thing with each one.

1. Make sure you have a variable that holds a table or list. It might come from a Question node, a tool, or a formula.
2. Add a Loop through items node.
3. In Select items to loop over, choose that variable.
4. Inside the loop, reference the current item to build a message, set a variable, or call an action.
5. After the loop, use the accumulated result — for example a single message summarising everything found.

Inside a loop, build a readable list:

`Set Topic.Summary = Concatenate(Topic.Summary, Topic.CurrentItem.Title, "; ")`

Or convert a table into a comma-separated string with Power Fx instead of looping at all, when you only need the text.

WATCH OUT

Dialog state does not fully reset just because you cleared variables or jumped back to the start node. If you need a genuinely repeatable flow — 'do you want to create another one?' — use a parent topic that asks the question and a child topic that does the work, and redirect between them. That reliably resets the conversation state.

TIP

Before you build a loop, ask whether Power Fx can do it in one line. Converting a table to a formatted string is usually faster, cheaper and easier to read than looping.

2.6 Switch Cases

Copilot Studio does not have a node called 'switch'. You build a switch by putting several branches on one Condition node, all testing the same variable. It is the right shape whenever one value decides which of many paths to take.

Pattern	What it looks like	Use when
Switch	One Condition node, many branches, all testing the same variable, plus 'All other conditions'.	One value picks one of several paths — department, leave type, request category.
Chain of conditions	Several Condition nodes one after another, each testing something different.	Several independent rules must all be evaluated.
Nested conditions	A Condition node inside a branch of another Condition node.	A second question only matters within one branch. Keep to two levels.

Pattern	What it looks like	Use when
Redirect to topic	A branch that hands off to a whole topic of its own.	A branch is big enough to deserve its own name and its own canvas.

[DIAGRAM 2.2]

Switch pattern: one variable, four branches — Finance, HR, IT, All other conditions — each redirecting to a dedicated topic.

BEST PRACTICE

When a switch grows past four or five branches, move each branch into its own topic and redirect. The canvas stays readable and each path can be tested on its own.

Convert choice variables to text with Text() before comparing, so branches match reliably.

Keep the branch order stable. Reordering branches silently changes behaviour when tests overlap.

Step-by-step Activity — Trace a value (10 minutes)

1. Open any topic that has a Question node.
2. Select the variable the question creates and note its current name and scope.
3. Rename it to something meaningful and save.
4. Add a Send a message node after the question, and insert the variable into the message text.
5. Test in the test canvas. Confirm the value you typed comes back in the message.
6. Now change the variable's scope to global and observe how its name changes from Topic. to Global.

Best Practices

- Name variables the way you would name a column in a spreadsheet somebody else has to read.
- Collect the minimum. Every extra question is a chance for the user to abandon the conversation.
- Prefer prebuilt entities, and let one sentence fill several slots at once.
- Give every branch a destination, including 'All other conditions'.
- Keep the canvas shallow. Deep nesting is the main reason topics become unmaintainable.

Common Mistakes

Mistake	Why it hurts	Do this instead
Leaving auto-generated variable names	Nobody, including future you, can read the topic.	Rename every variable as you create it.
Using global variables for everything	Values leak between topics and cause bugs that only appear in long conversations.	Default to topic scope; promote to global only when a second topic genuinely needs it.
Comparing a choice variable directly	The condition sees an object, not text, and silently never matches.	Wrap it in Text() or set a text variable first.
Empty 'All other conditions' branch	Users hit a silent dead end.	Always give it a message or a clarifying question.
Looping when Power Fx would do	Slower, harder to read, more to maintain.	Try a single formula first.
Asking questions the agent could infer	Long, annoying conversations.	Use entities and generative slot filling to extract from what the user already said.

Knowledge Check

Answer these on your own first, then compare with the answer key in Appendix C.

Q1. Which variable scope should you use for a value that only one topic needs?

- A. Global
- B. Topic
- C. System

D. Environment

Q2. What does slot filling mean in Copilot Studio?

- A. Adding buttons to a question.
- B. Placing an extracted entity value into a variable.
- C. Filling a knowledge source with documents.
- D. Reserving capacity for a conversation.

Q3. Your condition on a multiple-choice variable never matches, and the value looks like [object Object].

What is the fix?

- A. Change the variable to global scope.
- B. Wrap the variable in Text() before comparing, or set a text variable first.
- C. Delete and recreate the Question node.
- D. Switch the agent to classic orchestration.

Q4. Which statement about Memory in the new agent experience is TRUE?

- A. It is shared across all users of the agent.
- B. It replaces topic variables inside the authoring canvas.
- C. It maintains a separate memory store for each user and persists across conversations.
- D. It is only available for voice agents.

Q5. You need one value to decide which of four departments a request goes to. What is the cleanest pattern?

- A. Four separate topics with overlapping trigger phrases.
- B. One Condition node with four branches plus 'All other conditions', redirecting to a topic per department.
- C. A loop over the department list.
- D. Four nested Condition nodes.

Key Takeaways

- Variables hold values for one conversation; scope decides who can see them. Rename them the moment they appear.
- Entities decide what the agent looks for; slot filling is storing what it found. Prefer prebuilt entities.
- Memory is per-user and persists across conversations — a different tool from a variable, with privacy implications.
- Conditions branch, and the first true branch wins. Never leave 'All other conditions' empty.
- Loops repeat over a collection; a switch is one Condition node with several branches on the same variable.

Learn More — Microsoft Learn and Official Documentation

Microsoft Learn training

- **Work with entities and variables in Microsoft Copilot Studio (module)** — <https://learn.microsoft.com/en-us/training/modules/power-virtual-agents-entities/>
- **Create agents with Microsoft Copilot Studio — Online Workshop** — <https://learn.microsoft.com/en-us/training/paths/power-virtual-agents-workshop/>
- **Get started with Power Fx (learning path)** — <https://learn.microsoft.com/en-us/training/paths/use-basic-formulas-power-apps-canvas-app/>

Official documentation

- **Work with variables** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/authoring-variables>
- **Use entities and slot filling in agents** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/advanced-entities-slot-filling>
- **Additional settings for inputs of topics and actions** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/advanced-additional-settings-topic-action-inputs>
- **Memory in the new agent experience (preview)** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/agents-experience/memory-overview>
- **Use Power Fx in Copilot Studio** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/advanced-power-fx>
- **Power Fx formula reference** — <https://learn.microsoft.com/en-us/power-platform/power-fx/formula-reference>
- **Manage topics (loops, redirects, ending topics)** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/authoring-topic-management>

-
- **Environment variables in Power Platform** — <https://learn.microsoft.com/en-us/power-apps/maker/data-platform/environmentvariables>

Module 3 — Advanced Conversations

Duration: approximately 45 minutes. Related labs: Lab 12, Lab 13.

Learning Objectives

By the end of this module you will be able to:

- Design a multi-turn conversation that keeps context and survives interruptions.
- Make responses richer using formatting, quick replies and Adaptive Cards.
- Configure escalation to a human and explain when an agent should give up.
- Handle invalid answers and runtime errors so the user is never left stranded.
- Apply a short list of conversation design principles to any agent you build.

3.1 Multi-turn Conversations

A multi-turn conversation is any exchange that takes more than one round trip: the agent asks, the user answers, the agent uses that answer to decide what to ask next. Almost every useful agent is multi-turn.



Figure 3.1 — the five beats of a well-designed transaction. Skip 'Confirm' and you will regret it the first time a user mistypes a date.

Design concern	What to do	Where it lives
Keeping context	Store each answer in a well-named variable and refer back to it in later messages so the user can see the agent listened.	Module 2.1
Filling several slots at once	Use entities so one sentence can answer three questions. With generative orchestration, topic inputs can be filled from conversation context.	Module 2.2, 1.4
Interruptions	Users change their mind mid-flow. Trigger matching keeps running, so a message can jump to another topic. Design for it rather than fighting it.	Module 1.3
Reusing a step	Put the shared step in its own topic and use Redirect to another topic; by default the conversation returns afterwards.	Module 2.1
Ending cleanly	End current topic finishes this topic and returns to the caller. End all topics finishes everything.	This section
Starting fresh	End all topics does not clear global variables. Use a Clear all variables node first — the Reset Conversation system topic shows the pattern.	This section

[SCREENSHOT 3.1]

A multi-turn topic canvas showing three question nodes, a confirmation message and a condition before the final action.

BEST PRACTICE

Confirm before you act. Repeat back what you understood and give the user a way to correct it.
Ask one thing per question. Two questions in one message halve your completion rate.
Tell the user how many steps remain: 'Two quick questions and I can submit this.'
Let users bail out. A 'cancel' path in every transactional topic prevents the worst support tickets.

NOTE

For repeatable flows — 'would you like to raise another request?' — a parent topic that asks the question plus a child topic that does the work is more reliable than looping inside one topic, because redirecting between topics resets dialog state properly.

3.2 Adaptive Responses

A response does not have to be a wall of text. Copilot Studio gives you several ways to make answers easier to read and act on.

Technique	What it is	Good for
Message formatting	Bold, italics, lists, links, images and video in a Send a message node.	Making a long answer scannable.
Quick replies	Suggested reply buttons attached to a message.	Narrowing the next turn without a full question node.
Adaptive Card in a message	A rich card that displays information — a summary, a record, a status.	Showing structured results clearly.
Ask with Adaptive Card	An interactive card with input fields and one or more submit buttons. The values go straight into variables.	Collecting several fields in one turn instead of five questions.
Variable substitution	Inserting a variable into message text so the reply is personalised.	'Thanks {Global.EmployeeName}, I've logged three days of casual leave.'

[SCREENSHOT 3.2]

The Adaptive Card designer inside a topic, with the card preview on one side and the JSON payload on the other.

NOTE

Copilot Studio supports the Adaptive Cards schema up to version 1.6. You can design cards in the built-in designer, paste JSON, or build them in Power Apps or Power Automate and reuse the payload.

WATCH OUT

When you use an interactive card with submit buttons, make sure each submit action carries unique data identifying the card and the action. Without it, the agent cannot tell which card the user pressed.

BEST PRACTICE

Design cards for the narrowest channel you support. A card that looks good on the web can be unusable on a phone in Teams.

Follow the Adaptive Card accessibility guidance — screen readers and keyboard navigation matter, and Teams has its own considerations.

Do not put critical information only in a card. Some channels degrade cards to plain text.

Reuse and clone existing cards so your agent looks consistent.

3.3 Escalation

Escalation is the agent admitting it cannot help and moving the user on. Copilot Studio ships an **Escalate** system topic for exactly this. It runs when a message such as 'talk to agent' is matched, or when the Escalate event is called from anywhere in your agent.



Escalation trigger	How it happens	What to configure
User asks for a human	Their message matches the Escalate topic.	Rewrite the Escalate message in your own tone; include the real route to help.
A question keeps failing	A Question node can be set to escalate when it does not get a valid answer — this is the default behaviour.	Decide per question whether escalating or continuing is kinder.

Escalation trigger	How it happens	What to configure
A business rule says so	A branch in your topic redirects to Escalate — for example a high-value or compliance-sensitive request.	Make the rule explicit rather than leaving it to chance.
A live handoff is available	The agent transfers to a human in Dynamics 365 Customer Service or another engagement hub.	Configure handoff separately; the Escalate topic alone does not transfer the conversation.

BEST PRACTICE

Always give the user a real next step: a channel, a link, an email, or an actual transfer. 'Please contact support' with no detail is worse than saying nothing.

Pass context along. If you have collected an employee ID and a request type, include them in the handoff so the user does not repeat themselves.

Track escalation rate. A rising rate tells you which topic is failing before your users complain.

Escalate early on anything involving money, safety, legal exposure or an unhappy user.

3.4 Error Handling

Two different things go wrong in conversations, and they are handled in different places.

Kind of problem	Example	Where you handle it
The user gives an answer the agent cannot use	They type 'sometime next month' when the question needs a specific date.	The Question node's invalid-answer settings.
Something breaks at runtime	A tool call fails, or a value is missing where the topic expected one.	The On Error system topic.

Invalid answers — your three options

Option	What happens	Use it when
Escalate (default)	The user is redirected to the Escalate system topic.	The value is essential and a human is the right next step.
Set variable to value	The output variable is set to a default you choose, and the conversation continues.	A sensible default exists — 'Annual' leave, 'Standard' priority.
Set variable to empty	The variable is cleared and the conversation continues. You then use a Condition node to check whether it has a value.	You want to handle the gap yourself, later in the topic.

1. Open the Question node and go to its properties.
2. Find the settings for what happens when no entity value is found.
3. Choose Escalate, Set variable to value, or Set variable to empty.
4. Select Customize on the 'no entity found' message and write something helpful — say what a valid answer looks like, and give an example.
5. Test by typing something deliberately unusable, and confirm the user is not stranded.

Weak: "Sorry, I didn't get that."

Better: "I need a date I can work with. Try something like '15 August' or 'next Monday'. If you'd rather speak to someone, just say 'talk to a person'."

Runtime errors

The **On Error** system topic tells the user an error occurred. Its message includes an error code, the conversation ID and the timestamp — the three things your support team needs to trace what happened. Note that this system topic covers user-facing errors, not internal system errors.

BEST PRACTICE

Keep the error code, conversation ID and timestamp in the message. Removing them to make it look friendlier destroys your ability to debug.

Add a route out of the error — a link to the help desk or an offer to escalate.

Test the failure paths deliberately. Break a tool on purpose in your development environment and see what a user would experience.

3.5 Conversation Best Practices

Everything in this handbook reduces to a short list. Print this page.

Principle	In practice
Say what the agent can do	Open every conversation with two or three concrete examples, not 'How can I help you today?'
One thing per turn	One question, one idea, one action. Split anything with 'and' in it.
Be short	Three bullets or fewer, or two short sentences. Users skim.
Confirm before acting	Repeat what you understood and let the user correct it before anything is submitted.
Never dead-end	Every branch, every error and every failed match ends with a next step.
Offer the exit	'Talk to a person' should always work, at any point in any topic.
Be consistent	Same tone, same card style, same button labels across every topic.
Design for the channel	Test in Teams and on a phone, not only in the test canvas.
Be accessible	Meaningful link text, alt text on images, keyboard-navigable cards.
Close the loop	End with confirmation and a reference number the user can quote.

[DIAGRAM 3.1]

Conversation design checklist as a one-page visual: open, collect, confirm, act, close, with escalation and error paths branching from every stage.

Step-by-step Activity — Break your agent on purpose (10 minutes)

1. Open a topic with a Question node that uses a date or number entity.
2. In the test canvas, answer it with something meaningless.
3. Note exactly what the agent says and where the conversation goes.
4. Change the invalid-answer setting to Set variable to empty and add a Condition node that checks for a blank value.
5. Give the blank branch a helpful message with an example of a valid answer.
6. Re-test with the same meaningless answer and compare the experience.
7. Type 'talk to a person' and confirm the Escalate topic gives a real next step.

Best Practices

- Write the conversation as a script on paper before you open the canvas. If it reads badly aloud, it will feel worse in chat.
- Confirm, then act. Never submit anything on a single unconfirmed answer.
- Make escalation available from every topic, not just when the agent fails.
- Keep error messages diagnostic for support and human for the user — both, in the same message.
- Test every topic in the real channel; cards and formatting behave differently outside the test canvas.

Common Mistakes

Mistake	Why it hurts	Do this instead
No confirmation step	Users submit wrong data and blame the agent.	Repeat back and ask for a yes before acting.
Cards used for essential information only	Some channels downgrade cards to text and the user loses the message.	Repeat key facts in the message body.
Escalation with no route	'Please contact support' with no channel is a dead end.	Give a real link, address or live transfer.
Stripping error codes to look friendly	Support cannot trace the failure.	Keep code, conversation ID and timestamp.
Leaving invalid-answer handling at the default everywhere	Every fumbled answer escalates, and escalation volume explodes.	Choose per question: default value, blank plus a check, or escalate.
Testing only in the test canvas	Card rendering, authentication and formatting differ by channel.	Test in Teams and on a phone before release.

Knowledge Check

Answer these on your own first, then compare with the answer key in Appendix C.

Q1. Which node ends the current topic and returns to the topic that called it?

- A. End all topics
- B. End current topic
- C. Clear all variables
- D. Reset Conversation

Q2. You want to collect four fields in one turn instead of asking four questions. What should you use?

- A. A loop
- B. Ask with Adaptive Card
- C. Quick replies
- D. A global variable

Q3. A Question node does not get a valid answer. What is the default behaviour?

- A. The variable is set to empty and the topic continues.
- B. The user is redirected to the Escalate system topic.
- C. The On Error system topic runs.
- D. The conversation restarts.

Q4. Which three pieces of information does the On Error system topic message include?

- A. User name, channel and topic name
- B. Error code, conversation ID and timestamp
- C. Environment name, agent name and licence type
- D. Trigger phrase, entity and variable

Q5. You use End all topics but global variables still hold old values. What should you add?

- A. A Clear all variables node before it.
- B. A second End all topics node.
- C. A loop that resets each variable.
- D. A redirect to Conversation Start.

Key Takeaways

- Multi-turn design is five beats: trigger, collect, confirm, act, close — and confirmation is the one people skip.
- Adaptive Cards and formatting make answers usable; design for the narrowest channel and keep essentials in the message body.
- Escalation must always end in a real next step, and should be reachable from anywhere in the agent.
- Invalid answers and runtime errors are handled in different places: the Question node's settings, and the On Error system topic.
- Every branch, error and failed match needs a next step. Never dead-end a user.

Learn More — Microsoft Learn and Official Documentation

Microsoft Learn training

- **Deliver rich agent responses using Adaptive Cards in Copilot Studio (module)** — <https://learn.microsoft.com/en-us/training/modules/deliver-rich-agent-responses-adaptive-cards-copilot-studio/>
- **Enhance agents with Microsoft Copilot Studio (module)** — <https://learn.microsoft.com/en-us/training/modules/power-virtual-agents-enhance/>
- **Create agents in Microsoft Copilot Studio (learning path)** — <https://learn.microsoft.com/en-us/training/paths/create-extend-custom-copilots-microsoft-copilot-studio/>

Official documentation

- **Adaptive Cards overview** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/adaptive-cards-overview>
- **Ask with Adaptive Cards** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/authoring-ask-with-adaptive-card>
- **Use system topics (Escalate, Fallback, On Error)** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/authoring-system-topics>
- **Hand off conversations to a live agent** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/advanced-hand-off>
- **Additional settings for inputs of topics and actions** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/advanced-additional-settings-topic-action-inputs>
- **Manage topics — redirects, ending topics, clearing variables** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/authoring-topic-management>
- **Publish agents to channels and clients (guidance)** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/guidance/channels>
- **Analytics overview** — <https://learn.microsoft.com/en-us/microsoft-copilot-studio/analytics-overview>

Hands-on Labs — Day 2

Labs 7 to 11 extend the agent you built on Day 1. Labs 12 and 13 build two complete assistants that pull everything together. Work in order.

Lab	Title	You will produce
7	Create Topics	Two working topics with clean canvases.
8	Configure Trigger Phrases	Tested trigger phrases, plus a rewritten topic description.
9	Build Multi-turn Conversations	A three-question topic with confirmation and a cancel path.
10	Use Variables and Entities	Entity-driven questions filling well-named variables.
11	Implement Conditional Logic	A switch-style branch plus a loop over a list.
12	Design FAQ Assistant	A complete FAQ agent combining knowledge and topics.
13	Build Employee Leave Assistant	An end-to-end transactional assistant with escalation and error handling.

WATCH OUT

Labs 7 to 11 use the classic authoring experience, because that is where the topic canvas lives. If your tenant opens the new agent experience by default, switch it off using the New experience toggle before you start.

Lab 7 — Create Topics

Duration	Objective	Prerequisites
25 minutes	Create two topics from scratch and learn the canvas by building, not reading.	Day 1 agent open in the classic authoring experience.

Scenario

Your HR assistant needs two new abilities: telling employees the office holiday list, and starting a leave request. You will build the first one fully and stub the second.

Part 1 — A simple answer topic

1. Open your agent and go to Topics.
2. Select Add a topic, then From blank.
3. Name the topic 'Show holiday list'.
4. Add a Send a message node and write three or four holidays as a bulleted list.
5. Add an End current topic node.
6. Save, then open the test canvas and trigger the topic.
7. Open More, then Open code editor, and read the YAML version of what you just built.

Part 2 — A topic with a question

1. Create a second topic named 'Start leave request'.
2. Add an Ask a question node: 'Which type of leave would you like to apply for?'
3. Leave the Identify setting on multiple-choice options and add three options: Casual, Sick, Annual.
4. Rename the answer variable to LeaveType.
5. Add a Send a message node that repeats the choice back: 'Got it — a request for {LeaveType}.'
6. Add an End current topic node, save, and test.

[SCREENSHOT L7.1]

The 'Start leave request' topic canvas with a trigger, a question node showing three options, and a message node.

CHECKPOINT — YOU ARE DONE WHEN

Both topics appear on the Topics page and can be triggered in the test canvas.
The second topic repeats your choice back to you.
You have looked at one topic in the code editor.

If something goes wrong

Symptom	What to do
The topic does not trigger	In classic orchestration, check the trigger phrases. In generative orchestration, check the topic description.
The variable shows blank in the message	You inserted the wrong variable, or renamed it after inserting. Re-insert it from the variable picker.
The canvas will not save	An unconfigured node is usually the cause — look for a warning icon on a node.

Lab 8 — Configure Trigger Phrases

Duration	Objective	Prerequisites
25 minutes	Write trigger phrases that survive real users, and see for yourself what happens when two topics overlap.	Lab 7 complete.

Part 1 — Write good phrases

1. Open 'Show holiday list' and select the Trigger node.
2. Add these phrases, pressing Shift+Enter between each: 'when are the office holidays', 'holiday calendar for this year', 'is Friday a holiday', 'list of public holidays', 'which days is the office closed'.
3. Save and test using a sentence you did NOT type in — for example 'are we off next Thursday'.
4. Record whether it matched: _____

Part 2 — Break it on purpose

1. Open 'Start leave request' and add the single-word trigger phrase 'holiday'.
2. Save, then in the test canvas type 'holiday list please'.
3. Note which topic ran, or whether you were asked which topic you meant.
4. Remove the offending phrase and replace it with 'I want to apply for leave', 'raise a leave request', 'book time off', 'submit leave application'.
5. Re-test both topics and confirm each one wins for its own kind of question.

Part 3 — Write a description

1. Still on the Trigger node, write a description for 'Start leave request' using the pattern from Module 1.4: what it does, what it needs, and what it is NOT for.
2. Write your description here so your trainer can review it:

[SCREENSHOT L8.1]

The Trigger node with the final phrase list and the topic description filled in.

CHECKPOINT — YOU ARE DONE WHEN

Each topic has five to ten varied, realistic phrases.

You have seen the overlap problem happen and then fixed it.

'Start leave request' has a description that states what the topic is not for.

TIP

Keep every topic's phrases in one text file. Reading them side by side is the only reliable way to spot overlap before your users do.

Lab 9 — Build Multi-turn Conversations

Duration	Objective	Prerequisites
30 minutes	Turn a single question into a real transaction with confirmation and a way out.	Lab 8 complete.

Steps

1. Open 'Start leave request'.
2. After the leave-type question, add a second Ask a question node: 'What date does your leave start?' Set Identify to the Date and time prebuilt entity. Rename the variable StartDate.
3. Add a third question: 'How many days?' Set Identify to Number and rename the variable Days.
4. Add a Send a message node that summarises all three values in one sentence.
5. Add a fourth Ask a question node: 'Shall I submit this?' with options Yes and No. Rename the variable Confirmed.
6. Add a Condition node on Confirmed. On the Yes branch, send a confirmation message. On the 'All other conditions' branch, tell the user nothing was submitted and offer to start again.
7. Add End current topic on both branches.
8. Save and test the full flow twice — once saying yes, once saying no.



Figure L9.1 — the flow you are building.

Part 2 — Try an interruption

1. Start the topic again and answer the first question.
2. Instead of answering the second question, type 'when are the office holidays'.
3. Record what happened: _____
4. Discuss with your trainer whether that behaviour is right for this scenario.

[SCREENSHOT L9.1]

The completed multi-turn canvas with three questions, a summary message, a confirmation question and a two-branch condition.

CHECKPOINT — YOU ARE DONE WHEN

The topic collects three values and repeats them back before acting.

Saying No cancels cleanly rather than dead-ending.

You have observed what an interruption does mid-flow.

If something goes wrong

Symptom	What to do
The date question accepts anything	Identify is probably still on multiple-choice options. Change it to the Date and time entity.
The summary message shows the wrong value	Variables were renamed after being inserted. Re-insert them.
The condition never takes the Yes branch	Wrap the choice variable in Text() before comparing — see Module 2.1.

Lab 10 — Use Variables and Entities

Duration	Objective	Prerequisites
30 minutes	Replace button-picking with real language understanding, and pass a value between topics.	Lab 9 complete.

Part 1 — A custom closed-list entity

1. In the left navigation, open Entities and select New entity, then Closed list.
2. Name it 'Department' and add four items: Finance, HR, IT, Operations.
3. For each item, add synonyms — for Finance add 'accounts' and 'fin'; for IT add 'tech' and 'helpdesk'.
4. Save the entity.
5. In 'Start leave request', add a question 'Which department are you in?' and set Identify to your Department entity.
6. Rename the variable Department and test by typing 'I'm in accounts' rather than selecting a button.

Part 2 — A regular-expression entity

1. Create a second entity, this time Regular expression, named 'Employeeid'.
2. Use a pattern that matches your organisation's format — for example EMP followed by a hyphen and five digits.
3. Add a question 'What is your employee ID?' using this entity, and include an example in the question text.
4. Test with a valid ID and then with an invalid one, and note the difference.

Part 3 — Pass a value between topics

1. Change the Department variable's scope so it can receive values from other topics.
2. Create a small topic named 'Get department' that asks only that one question.
3. In 'Start leave request', replace the department question with a Redirect to another topic node pointing at 'Get department'.
4. Use Add input for destination topic to pass and return the value.
5. Test the whole flow and confirm the department still reaches the summary message.

[SCREENSHOT L10.1]

The custom Department closed-list entity with synonyms, alongside a question node using it.

CHECKPOINT — YOU ARE DONE WHEN

Typing 'accounts' is recognised as Finance.
 An invalid employee ID is rejected with a helpful message.
 A value collected in one topic is used in another.

If something goes wrong

Symptom	What to do
Synonyms are not recognised	Check they were added to the right list item and the entity was saved.
The value does not arrive in the calling topic	The variable scope was not changed to receive values from other topics, or no input was mapped on the redirect node.
The regular expression rejects everything	Test the pattern outside Copilot Studio first; anchoring and case sensitivity are the usual culprits.

Lab 11 — Implement Conditional Logic

Duration	Objective	Prerequisites
30 minutes	Build a switch-style branch and a loop, and see when Power Fx beats both.	Lab 10 complete.

Part 1 — The switch

1. In 'Start leave request', add a Condition node after the Department question.
2. Create one branch per department, each testing `Text(Topic.Department)` against the department name.
3. Give each branch a different message naming the approver for that department.
4. Fill the 'All other conditions' branch with a message that asks the user to pick a department from the list.
5. Save and test all five paths, including an unrecognised department.

Part 2 — A rule with Power Fx

1. Add a second Condition node after the Days question.
2. Switch the branch to a formula and enter: `Topic.Days > 5`
3. On the true branch, tell the user their request needs manager approval.
4. On the other branch, tell them it will be approved automatically.
5. Test with 4, 5 and 6 days and confirm the boundary behaves as you expect.

Part 3 — A loop

1. Add a Set a variable value node that creates a small table of example pending requests, or use a table your trainer supplies.
2. Add a Loop through items node and select that table in Select items to loop over.
3. Inside the loop, build up a summary string using Concatenate and the current item.
4. After the loop, send one message containing the whole summary.
5. Then try the alternative: replace the loop with a single Power Fx expression that joins the table into a comma-separated string, and compare the two approaches.

Inside the loop:

```
Set Topic.Summary = Concatenate(Topic.Summary, Topic.CurrentItem.Title, "; ")
```

Boundary tests for Part 2: enter 4, then 5, then 6.

CHECKPOINT — YOU ARE DONE WHEN

Every department branch works, including the fallback branch.

The five-day boundary behaves exactly as intended.

You have built the same summary twice — once with a loop, once with a formula — and can say which you would ship.

Reflection

- At what point would you move each department branch into its own topic?
- Which of your branches would a new colleague misread?

Lab 12 — Design FAQ Assistant

Duration	Objective	Prerequisites
35 minutes	Build a complete FAQ assistant that knows when to answer from knowledge and when to run a topic.	Labs 7 to 11 complete.

Scenario

Contoso's IT helpdesk answers the same twenty questions every week. Most are pure information — knowledge can handle them. Three need a specific, controlled answer that must never vary.

Steps

1. Create a new agent named 'IT FAQ Assistant — <your initials>', or a new set of topics inside your existing agent if your trainer prefers.
2. Attach one knowledge source: an IT policy document or a public support page.
3. Open Conversation Start and rewrite the greeting so it names three things the agent can actually do.
4. Test five general questions and confirm Conversational boosting answers them with citations.
5. Now build three controlled topics: 'Reset password', 'Request software' and 'Report a laptop fault'. Each needs a trigger or description, one or two questions, a confirmation and a clear close.
6. Rewrite the Fallback system topic so an unmatched question offers the three topics by name.
7. Rewrite the Escalate system topic so it gives the real helpdesk route.
8. Run the full test script: five knowledge questions, three topic questions, one ambiguous question, one out-of-scope question.

Your test results

#	Question you asked	Answered by knowledge or topic?	Correct?	Notes
1	_____	Knowledge / Topic	Y / N	_____
2	_____	Knowledge / Topic	Y / N	_____
3	_____	Knowledge / Topic	Y / N	_____
4	_____	Knowledge / Topic	Y / N	_____
5	_____	Knowledge / Topic	Y / N	_____
6	_____	Knowledge / Topic	Y / N	_____

[SCREENSHOT L12.1]

The IT FAQ Assistant answering a knowledge question with a citation, next to a controlled topic running.

CHECKPOINT — YOU ARE DONE WHEN

General questions are answered from knowledge, with citations.
The three controlled topics run reliably and never vary their wording.
Fallback and Escalate both name real next steps.

BEST PRACTICE

Use topics for anything that must be word-for-word identical every time — compliance answers, security instructions, refund rules.
Use knowledge for the long tail. Building a topic per question does not scale and the answers drift apart.

Lab 13 — Build Employee Leave Assistant

Duration	Objective	Prerequisites
40 minutes	Assemble everything from Day 2 into one production-shaped assistant.	Labs 7 to 12 complete.

Scenario

You will finish the leave assistant properly: it answers policy questions, collects a request, branches on business rules, confirms before acting, handles bad input, and escalates when it should.



Steps

1. Start from the agent you have been extending. Confirm the HR policy knowledge source from Day 1 is still attached and working.
2. Rewrite Conversation Start to name the three things this assistant does: answer leave policy questions, start a leave request, and check a request's status.
3. Bring together the work from Labs 9, 10 and 11 into a single 'Start leave request' topic: leave type, department, start date, number of days, confirmation.
4. Add the business rule branch: more than five days needs manager approval.
5. Set the invalid-answer behaviour on the date question to Set variable to empty, then add a Condition node that checks for blank and asks again with an example.
6. Add a 'talk to a person' path: make sure the Escalate topic gives the HR helpdesk address and passes along the employee's department.
7. Add an Adaptive Card, or a well-formatted message, that summarises the request before the confirmation question.
8. Rewrite the On Error message so it keeps the error code, conversation ID and timestamp but also offers a route to a human.
9. Test the eight scenarios in the table below.
10. Publish the agent and repeat two of the tests inside Teams or Microsoft 365 Copilot.

Acceptance tests

#	Scenario	Expected result	Pass?
1	Ask a leave policy question	Answered from knowledge, with a citation	Y / N
2	Apply for 3 days casual leave	Collected, summarised, confirmed, closed	Y / N
3	Apply for 8 days annual leave	Manager-approval branch taken	Y / N
4	Give a nonsense start date	Helpful re-prompt with an example, not an escalation	Y / N
5	Say 'I'm in accounts'	Recognised as Finance	Y / N
6	Answer No at the confirmation step	Nothing submitted, clean exit offered	Y / N
7	Type 'talk to a person' mid-flow	Escalate topic with a real HR route	Y / N
8	Ask something completely unrelated	Fallback offers the three things the agent can do	Y / N

[SCREENSHOT L13.1]

The Employee Leave Assistant in Teams, showing the summary card and the confirmation buttons.

CHECKPOINT — YOU ARE DONE WHEN

All eight acceptance tests pass.

The agent is published and two tests have been repeated in the real channel.

You can explain, for any failing test, whether the cause was a topic, an entity, a condition or an error setting.

Reflection

- Which of the eight tests failed first, and what did that tell you about your design?
- What would you add before letting real employees use this — and what would you deliberately leave out of version one?
- Which parts of this assistant would you rebuild in the new agent experience, and which would you keep as topics?

Appendix A — Day 2 Vocabulary

New terms introduced today. Day 1 terms are in the Day 1 handbook.

Term	Meaning
Adaptive Card	A platform-agnostic card of text, graphics, inputs and buttons. Copilot Studio supports schema 1.6 and earlier.
Closed list entity	A custom entity built from a fixed list of values, each with synonyms.
Condition node	A node that splits the conversation into branches; the first true branch runs.
Dynamic entity	Options supplied at runtime from a list variable, rather than fixed when you authored the topic.
Entity	A definition of what kind of information to look for in a user's reply.
Environment variable	A value that differs between development, test and production, referenced as Env.
Generative orchestration	The default mode where a planning layer chooses topics, tools and knowledge based on names and descriptions.
Classic orchestration	The mode where the topic whose trigger phrases best match the message is the one that runs.
Global variable	A variable that lives for the whole conversation and is visible to every topic.
Loop through items	A node that repeats the same steps once for each item in a list or table.
Memory	A new-experience capability that lets an agent remember details per user across conversations. In preview.
Node	One step on the topic canvas — message, question, condition, loop, action, redirect or end.
Power Fx	The formula language used in conditions and variable expressions across Power Platform.
Quick reply	A suggested reply button attached to a message.
Redirect	A node that hands the conversation to another topic and, by default, returns afterwards.
Slot filling	Placing an extracted entity value into a variable.
Switch pattern	One Condition node with several branches, all testing the same variable.
System topic	A built-in topic added automatically — Conversation Start, Fallback, Escalate, On Error and others.
System variable	Read-only context supplied by the platform, referenced as System.
Topic	A self-contained piece of conversation logic with a trigger and a set of nodes.
Trigger phrase	An example message used in classic orchestration to decide which topic runs.
YAML code editor	The text view of a topic, useful for copying topics and reviewing changes.

Appendix B — Day 2 Quick Reference

Node cheat sheet

I want to...	Use this node
Say something	Send a message
Ask for one value	Ask a question (+ choose an entity under Identify)
Ask for several values at once	Ask with Adaptive Card
Branch the conversation	Add a condition
Repeat over a list	Loop through items
Store or calculate a value	Set a variable value (with Power Fx)
Reuse another topic	Redirect to another topic
Finish this topic	End current topic
Finish everything	End all topics (preceded by Clear all variables if needed)

Power Fx snippets you will reuse

<code>Text(Topic.Choice)</code>	make a choice comparable
<code>IsBlank(Topic.Value)</code>	check nothing was captured
<code>Concatenate(a, b)</code>	join strings
<code>Lower(Trim(Topic.Input))</code>	normalise before comparing
<code>And(x, y) Or(x, y) Not(x)</code>	combine tests
<code>If(Topic.Days > 5, "Approval", "Auto")</code>	inline branching
<code>CountRows(Topic.Table)</code>	size of a collection
<code>DateDiff(Topic.Start, Topic.End, Days)</code>	days between two dates

Trigger phrase rules

- Five to ten phrases, not fifty.
- Whole sentences the way users actually type them.
- Vary length, wording and formality.
- Include real misspellings and shorthand from support tickets.
- No single keywords, and no overlap between topics.

The five beats of a transaction



Never dead-end checklist

Place users get stuck	What must be there
'All other conditions' branch	A message or a clarifying question
Invalid answer to a question	A re-prompt with an example, or a sensible default
No topic matched	Fallback that names what the agent can do
Two topics matched	Distinct trigger phrases so this stops happening
Runtime error	Error code, conversation ID, timestamp — plus a route to a human
User asks for a person	Escalate with a real channel, link or live transfer

Appendix C — Knowledge Check Answer Key

Module 1 — Topics

Q	Answer	Why
1	B	In generative orchestration the default trigger is 'The agent chooses', driven by the topic name and description.
2	C	Conversational boosting contains the generative answers node and is created automatically.
3	B	Multiple Topics Matched is the 'did you mean' prompt shown when a message matches several topics.
4	B	Realistic whole sentences generalise well; single keywords collide across topics.
5	B	Copilot Studio generates a default description for each topic from its trigger phrases — a starting point you should rewrite.

Module 2 — Conversation Design

Q	Answer	Why
1	B	Topic scope keeps the value local; promote to global only when another topic genuinely needs it.
2	B	Slot filling means placing an extracted entity value into a variable.
3	B	A choice variable is an object; wrap it in Text() or set a text variable before comparing.
4	C	Memory keeps a separate store per user and persists across conversations. It is in preview in the new experience.
5	B	One Condition node with a branch per department, plus 'All other conditions', is the switch pattern.

Module 3 — Advanced Conversations

Q	Answer	Why
1	B	End current topic finishes this topic and returns to the caller; End all topics finishes everything.
2	B	Ask with Adaptive Card collects several fields in one turn and stores them in variables.
3	B	Escalate is the default behaviour when a Question node does not get a valid answer.
4	B	The On Error message includes an error code, the conversation ID and the timestamp.
5	A	End all topics does not clear global variables; add a Clear all variables node first.

Appendix D — Notes and Next Steps

Use this space for your own notes, questions for the trainer, and design ideas for your own agents.

One conversation I will redesign at work

Question	Your answer
Which process today needs too many emails?	
What are the three questions you would have to ask?	
What would the confirmation message say?	
When should it escalate to a human?	

Coming up on Day 3

- Tools and actions — connecting agents to Power Automate, connectors and REST APIs.
- Agent flows and autonomous triggers.
- Multi-agent systems and connected agents.
- Evaluation, analytics and continuous improvement.
- Application lifecycle management: solutions, pipelines and moving agents to production.